

MATRIXPOLICY: ROLE-AWARE OPTIMIZATION FOR RATIONAL TRANSFORMER NONLINEARITIES

Anonymous authors

Paper under double-blind review

ABSTRACT

Fixed token budgets make Transformer pretraining an arrival-time problem: beyond endpoint loss, the practical question is how many tokens are needed to reach a useful validation loss. Most activation design changes the forward response of the nonlinear sublayer, from pointwise curves to gated forms, while the optimizer still sees the surrounding matrices through a generic tensor interface. We study a different lever: expose the sublayer’s matrix roles to the optimizer. Rational Latent Basis (RLB) maps residual states into normalized groups and learns real-pole-free P5/Q4 rational response curves that provide response and matrix-pair scale summaries. MatrixPolicy uses these summaries to update the RLB input and output matrices by role, while AdamW handles rational coefficients and all remaining Transformer parameters. Across five 12-layer, width-768 language-model pretraining corpora, MatrixPolicy reaches common validation-loss targets earlier than matched SiLU and RLB controls. At 100M tokens it saves 19.5–22.8% target-arrival tokens versus SiLU with AdamW and 29.2–34.0% versus SiLU with Muon; at 300M tokens, after endpoint losses begin to saturate, it still saves 22.3–26.4% and 6.7–9.8%, respectively, with lower measured arrival time in every listed dataset/comparator pair. These results support designing activations as optimizer interfaces for token-efficient training. [todo: Add the larger-scale training result once the next experiment is complete.]

1 INTRODUCTION

A central cost in language-model pretraining is the number of tokens required to reach a useful validation loss. The nonlinear sublayer is a natural target for reducing this cost because it is a structured computation: an input matrix selects latent coordinates, a response transforms them, and an output matrix recombines the resulting features. Much activation design has improved the forward response. Pointwise activations such as GELU and SiLU choose fixed scalar curves, while gated sublayers such as GLU and SwiGLU add multiplicative structure and became prominent in modern language models (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020). In these sublayers, the scalar nonlinearity is embedded between learned projections, gates, and output recombination, so adjacent matrices can absorb scale and remix feature directions. Isolated curve changes are therefore a limited lever. The optimizer, however, still updates the surrounding matrices through a generic parameter-gradient interface.

This leaves a different lever: change what the optimizer can see about the nonlinear sublayer. The input matrix controls which latent directions enter each response group, the rational response shapes the transformed values, and the output matrix recombines those values into the residual stream. Treating these matrices as generic tensors hides this role asymmetry and the reciprocal scale balance between them. We take this route with Rational Latent Basis (RLB), a grouped rational sublayer that reports normalized group coordinates, trainable rational response curves, derivative/output response scales, and a positive matrix-pair scale relation. The evaluated RLB variant is global-rational: it uses one trainable real-pole-free P5/Q4 response per group and projects the transformed groups back to the model width.

MatrixPolicy is the role-aware update rule paired with RLB. For the RLB input and output matrices, it uses the exposed response and scale summaries to perform centered group-gradient redistribution, a transient gated Muon substep, and bounded matrix-pair rescaling. Rational coefficients, attention

weights, embeddings, and all remaining Transformer parameters use AdamW in the reported configuration. This separation keeps the optimizer change concentrated on the two matrices whose roles are made explicit by the sublayer.

We evaluate the method on a fixed 12-layer, width-768 causal Transformer across DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4. The 100M-token budget captures early optimization behavior, where MatrixPolicy reaches shared validation-loss targets substantially earlier than AdamW and Muon comparators on matched SiLU and RLB controls. The 300M-token budget tests the same architecture after longer training, where endpoint gaps narrow but the target-arrival advantage remains visible in both tokens and measured time. [todo: Add the larger-scale experiment result here once the next run is complete.]

The contributions are:

- a global-rational RLB sublayer that exposes normalized groups, rational response summaries, and matrix-pair scale information to the optimizer;
- MatrixPolicy, a role-aware update rule for the RLB input and output matrices that combines group-gradient redistribution, a gated matrix substep, and bounded pair rescaling while AdamW handles the remaining parameters;
- matched evidence across five pretraining corpora showing faster target arrival in tokens and measured time against AdamW and Muon controls, with longer-budget runs measuring the saturation regime.

2 RELATED WORK

Transformer nonlinear sublayers are often identified by their scalar response, from GELU and Swish/SiLU to gated variants such as GLU and SwiGLU (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020). RLB follows the same architectural location but changes the interface exposed to optimization: instead of replacing only a fixed scalar curve, it exposes grouped coordinates, trainable rational responses, and the adjacent matrix roles used by MatrixPolicy.

Rational activations are motivated by the ability of low-degree rational functions to represent sharp transitions, saturation, and curved response shapes with few parameters. Recent approximation-theoretic work argues that trainable low-degree rational activations can have expressivity advantages over common fixed activations (Tang & Townsend, 2026). We use that result as motivation for the activation family, not as a proof of MatrixPolicy. The empirical comparison separates the global-rational RLB sublayer from the role-aware optimizer by including RLB controls with AdamW, Muon, Lion, SOAP, CAME, and ScheduleFree-style optimizers.

Adam and AdamW remain standard Transformer pretraining optimizers (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). Other work changes the first-order rule while keeping a generic tensor interface, including Lion and cautious optimizers (Chen et al., 2023; Liang et al., 2026), or uses matrix structure and low-rank information, including Muon, Sophia, GaLore, SOAP, and Adamini (Liu et al., 2025; 2024; Zhao et al., 2024; Vyas et al., 2025; Zhang et al., 2025). CAME and schedule-free training modify optimizer state or schedules (Luo et al., 2023; Defazio et al., 2024). MatrixPolicy is closest in spirit to matrix-aware optimization, but it does not infer role from the parameter tensor alone: it receives activation-specific response gains, group-pressure signals, and pair-balance summaries from RLB. In the reported configuration, role-specific rules apply only to the RLB input and output matrices; rational coefficients and non-RLB-matrix Transformer parameters use AdamW. Broader optimizer coverage is available in recent surveys (Wen et al., 2026).

3 METHOD

RLB and MatrixPolicy are designed as a single interface. RLB defines the nonlinear sublayer and the detached summaries it exposes; MatrixPolicy consumes those summaries to update the two adjacent matrices. Rational coefficients and ordinary Transformer parameters remain on the AdamW path in the reported configuration.

Throughout the method we use

$$\text{clip}(x, a, b) = \min\{\max\{x, a\}, b\}, \quad (1)$$

$$\text{smoothstep}(a, b, x) = 3\omega^2 - 2\omega^3, \quad \omega = \text{clip}\left(\frac{x-a}{b-a}, 0, 1\right). \quad (2)$$

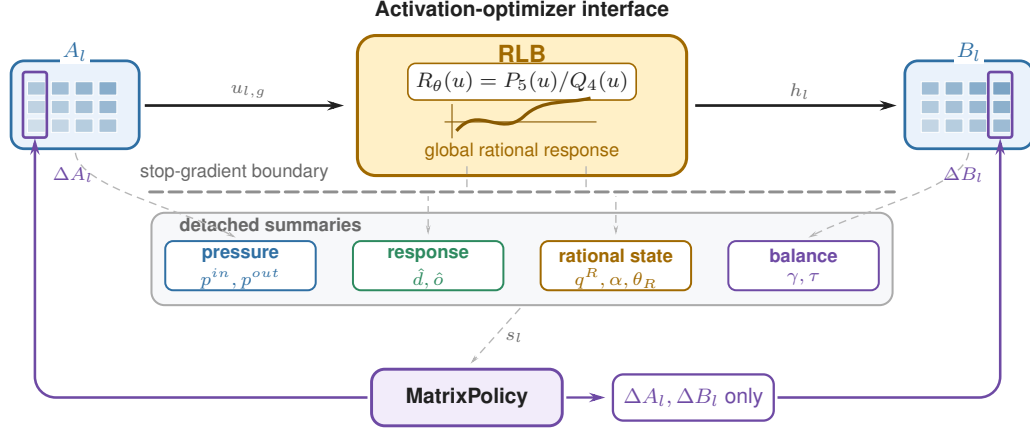


Figure 1: Activation-optimizer interface for global-rational RLB. In layer l , A_l maps the sublayer input into normalized rational groups and B_l recombines the rational features. RLB exposes matrix-pressure, response-gain, coefficient-activity, and pair-balance summaries through a stop-gradient boundary. MatrixPolicy reads these summaries and writes only ΔA_l and ΔB_l ; rational coefficients are AdamW-trained and policy-observed, not MatrixPolicy update targets.

3.1 RATIONAL LATENT BASIS

For layer $l \in \{1, \dots, L\}$, let $x_l \in \mathbb{R}^d$ be the input to the nonlinear sublayer. RLB projects x_l to a latent width $H = Gm$, partitions the latent coordinates into G groups, normalizes each group by its RMS radius, and applies one trainable group-global rational response:

$$z_l = A_l x_l, \quad z_l = \text{concat}_{g=1}^G z_{l,g}, \quad z_{l,g} \in \mathbb{R}^m, \quad A_l \in \mathbb{R}^{H \times d}, \quad (3)$$

$$r_{l,g} = \sqrt{m^{-1} \|z_{l,g}\|_2^2 + \epsilon_{\text{rms}}}, \quad u_{l,g} = z_{l,g} / r_{l,g}, \quad (4)$$

$$P_{l,g}(u) = \sum_{i=0}^5 a_{l,g,i} u^i, \quad (5)$$

$$Q_{l,g}(u) = 1 + |b_{l,g,1}| |u| + |b_{l,g,2}| u^2 + |b_{l,g,3}| |u|^3 + |b_{l,g,4}| u^4, \quad (6)$$

$$R_{l,g}(u) = P_{l,g}(u) / Q_{l,g}(u), \quad h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \quad (7)$$

$$y_l = B_l \text{concat}_{g=1}^G h_{l,g}, \quad B_l \in \mathbb{R}^{d \times H}.$$

Here $\epsilon_{\text{rms}} > 0$ is the stabilizing squared-RMS floor; the rescaling identity below explicitly sets this floor to zero. The scalar functions $P_{l,g}$, $Q_{l,g}$, and $R_{l,g}$ are applied coordinatewise to vector inputs. The denominator satisfies $Q_{l,g}(u) \geq 1$ for real u , so the implemented curve has no real pole. The response is piecewise rational because of the absolute-value terms; derivative statistics use the autodiff derivative of this implemented expression, with $\text{sign}(0) = 0$, and equations involving $R'_{l,g}$ are interpreted almost everywhere. Numerator and denominator coefficients are initialized by a SiLU fit on $[-5, 5]$ and are then trained by AdamW. The reported model uses only these group-global P5/Q4 rational responses.

The design separates where the response is evaluated from how much group scale is returned to the residual stream. The normalized coordinate $u_{l,g}$ indexes the rational curve, while $r_{l,g}$ carries the

latent group scale. For fixed curves $R_l = \{R_{l,g}\}_{g=1}^G$, write the block map

$$f_l(x; A_l, B_l, R_l) = B_l \text{concat}_{g=1}^G r_{l,g} R_{l,g}(u_{l,g}), \quad (8)$$

with $z_l = A_l x$ and $r_{l,g}, u_{l,g}$ defined as above. If $\epsilon_{\text{rms}} = 0$ and every group radius is nonzero, then for any $\lambda \in \mathbb{R}_{>0}^G$ and $D_l(\lambda) = \text{diag}(\lambda_1 I_m, \dots, \lambda_G I_m)$,

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(\lambda) A_l, B_l D_l(\lambda)^{-1}, R_l). \quad (9)$$

This positive matrix-pair rescaling is exact only in the unfloored idealization. With the stabilizing RMS floor and optimizer state, MatrixPolicy treats it as a bounded balancing direction rather than an exact invariance of the implemented training dynamics.

3.2 MATRIXPOLICY

MatrixPolicy acts only on the RLB matrix pair (A_l, B_l) . It has three design operations. First, it redistributes update scale across rational groups using live response gains and relative matrix-gradient pressure. Second, it adds a transient matrix-aware substep during early training, when endpoint saturation has not yet compressed method differences. Third, it performs bounded reciprocal rescaling of each input/output matrix pair along the positive balance coordinate exposed by RLB. At step t , these operations read gradients and cached RLB response summaries from the current minibatch, rescale matrix gradients by group and role, apply the AdamW/Muon matrix branch, and finally perform the scheduled positive matrix-pair balancing move. Unless stated otherwise, gradient-derived statistics in this subsection are computed after backpropagation and before any parameter update. The final pair-balance operation is applied after the AdamW/Muon matrix branch and computes its current matrix ratio at that point. For any tensor V , define

$$\text{rms}(V) = \sqrt{\text{mean}(V^2)}, \quad \text{rms}_\epsilon(V) = \sqrt{\text{mean}(V^2) + \epsilon}. \quad (10)$$

The constants $\epsilon_p > 0$ and $\epsilon_g > 0$ denote fixed additive floors in squared-RMS units. The same ϵ_p is reused as the implementation floor for matrix-ratio logs in equation 29. The resulting pressure and activity scores are bounded update-control signals. Let $A_{l,g} \in \mathbb{R}^{m \times d}$ be the row block of A_l , and let $B_{l,g} \in \mathbb{R}^{d \times m}$ be the column block of B_l . The relative matrix-gradient scales are

$$p_{l,g}^{\text{in}} = \frac{\text{rms}_{\epsilon_p}(\nabla A_{l,g})}{\text{rms}_{\epsilon_p}(A_{l,g})}, \quad p_{l,g}^{\text{out}} = \frac{\text{rms}_{\epsilon_p}(\nabla B_{l,g})}{\text{rms}_{\epsilon_p}(B_{l,g})}. \quad (11)$$

The coefficient-gradient energy scale for the global response is

$$p_{l,g}^R = \left[\frac{1}{2} (\text{mean}((\nabla a_{l,g,\cdot})^2) + \text{mean}((\nabla b_{l,g,\cdot})^2)) + \epsilon_p \right]^{1/2}. \quad (12)$$

The same numerical floor is used in $p_{l,g}^R$, although this quantity is raw coefficient-gradient energy rather than a relative matrix pressure. MatrixPolicy maintains exponential moving averages $q_{l,g}^{\text{in}}$, $q_{l,g}^{\text{out}}$, and $q_{l,g}^R$ of these quantities with decay 0.95, and forms

$$\pi_{l,g} = \log q_{l,g}^{\text{in}} - \log q_{l,g}^{\text{out}}, \quad \alpha_{l,g} = \log q_{l,g}^R - \frac{1}{2} (\log q_{l,g}^{\text{in}} + \log q_{l,g}^{\text{out}}). \quad (13)$$

Here each matrix pressure is an unsigned RMS relative-gradient scale. Consequently $\pi_{l,g}$ is a signed log imbalance between two unsigned magnitudes: its sign records which role currently has the larger RMS-relative scale. MatrixPolicy uses this imbalance only as a bounded control signal for redistribution and pair balancing. The score $\alpha_{l,g}$ compares coefficient-gradient activity with adjacent matrix pressure and is used only inside bounded redistribution and gating rules.

The activation also provides batch-estimated curve gains. From a recent sample of normalized coordinates $u_{l,g}$ cached by the RLB forward path, MatrixPolicy uses floored summaries

$$\hat{d}_{l,g}^{\text{live}} = \sqrt{\text{mean}(R'_{l,g}(u_{l,g})^2) + \epsilon_g}, \quad \hat{o}_{l,g}^{\text{live}} = \sqrt{\text{mean}(R_{l,g}(u_{l,g})^2) + \epsilon_g}. \quad (14)$$

In equation 14, the mean is over all cached scalar normalized coordinates for group (l, g) across batch, sequence positions, and the m group coordinates. The derivative gain is a proxy for the

input-domain role. The output gain is a curve-response proxy for the recombination role; it does not include the group radius $r_{l,g}$ and therefore is not the full feature scale seen by B_l . Both gains are cheap role summaries rather than full block-Jacobian preconditioners.

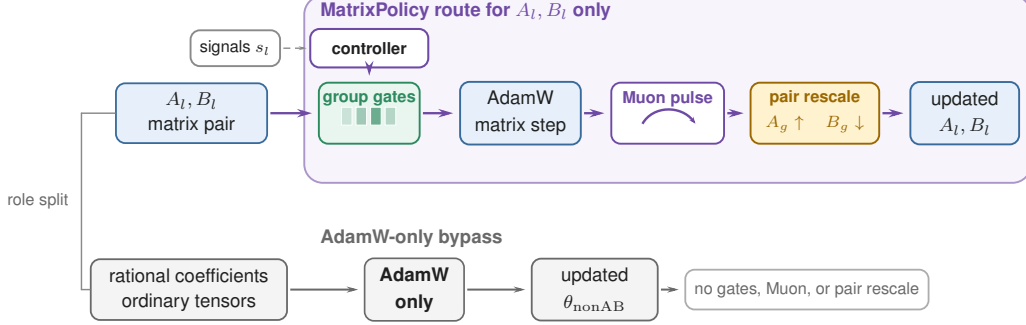


Figure 2: Selective MatrixPolicy route for the reported optimizer. Only the RLB matrix pair A_l, B_l enters this route: policy signals set group gradient gates $c_{l,g,\sigma}$, AdamW supplies the base matrix step, a transient Muon correction acts on the same matrices, and pair rescaling applies reciprocal updates $A_{l,g} \leftarrow e^\ell A_{l,g}$ and $B_{l,g} \leftarrow e^{-\ell} B_{l,g}$ to move the current balance γ toward the target τ . Rational coefficients and all non-RLB matrix parameters bypass this route and remain AdamW-only.

Let t be the optimizer step, T the planned number of training steps, and $\xi_t = t/T$. A smooth gate

$$\phi_t = \text{smoothstep}(0.02, 0.30, \xi_t) \quad (15)$$

turns on the group-gradient policy. For positive group values v_j , define $\text{geomean}_{j=1}^G(v_j) = \exp(G^{-1} \sum_{j=1}^G \log v_j)$. The multiplier for group g and role $\sigma \in \{\text{in}, \text{out}\}$ is

$$c_{l,g,\sigma} = c_{l,g,\sigma}^{\text{gain}} c_{l,g,\sigma}^{\text{pressure}} c_{l,g}^{\text{activity}}. \quad (16)$$

For the input role, the gain term uses $\hat{d}_{l,g}^{\text{live}}$; for the output role, it uses $\hat{o}_{l,g}^{\text{live}}$:

$$c_{l,g,\text{in}}^{\text{gain}} = \left(\frac{\text{geomean}_{j=1}^G \hat{d}_{l,j}^{\text{live}}}{\hat{d}_{l,g}^{\text{live}}} \right)^{0.20\phi_t}, \quad c_{l,g,\text{out}}^{\text{gain}} = \left(\frac{\text{geomean}_{j=1}^G \hat{o}_{l,j}^{\text{live}}}{\hat{o}_{l,g}^{\text{live}}} \right)^{0.20\phi_t}. \quad (17)$$

With $\tilde{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.50, 1.50)$, the relative-gradient term sets role-dependent group multipliers before the later role-wise recentering:

$$c_{l,g,\text{in}}^{\text{pressure}} = \exp(-0.10\phi_t \tilde{\pi}_{l,g}), \quad c_{l,g,\text{out}}^{\text{pressure}} = \exp(+0.10\phi_t \tilde{\pi}_{l,g}). \quad (18)$$

Coefficient-gradient activity downweights groups whose rational coefficients are moving more strongly than the adjacent matrices before role-wise recentering:

$$c_{l,g}^{\text{activity}} = \exp \left[-0.20\phi_t \text{ReLU} \left(\frac{\alpha_{l,g} - 0.05}{0.45} \right) \right]. \quad (19)$$

The final multiplier is recentered and clipped,

$$c_{l,g,\sigma} \leftarrow \frac{c_{l,g,\sigma}}{\text{geomean}_{j=1}^G c_{l,j,\sigma}}, \quad c_{l,g,\sigma} \leftarrow \text{clip}(c_{l,g,\sigma}, 0.75, 1.35), \quad (20)$$

so the policy redistributes update scale across groups, separately for each matrix role, without changing the role-wise mean log multiplier before clipping.

The scaled matrix gradients are

$$\tilde{\nabla} A_{l,g} = c_{l,g,\text{in}} \nabla A_{l,g}, \quad \tilde{\nabla} B_{l,g} = c_{l,g,\text{out}} \nabla B_{l,g}. \quad (21)$$

Let $\tilde{\nabla} M_{l,\text{in}}$ be the row-block concatenation of $\{\tilde{\nabla} A_{l,g}\}_{g=1}^G$, and let $\tilde{\nabla} M_{l,\text{out}}$ be the column-block concatenation of $\{\tilde{\nabla} B_{l,g}\}_{g=1}^G$. After this scaling, the RLB matrices are updated by a role/depth AdamW branch and a transient Muon branch. Define $\delta_l = (l-1)/(L-1)$ for $L > 1$ and $\delta_l = 1/2$ otherwise. The role factors are

$$\varrho_{\text{in}}(l) = \text{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \quad \varrho_{\text{out}}(l) = \text{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40), \quad (22)$$

and the AdamW matrix multiplier for role σ is

$$s_{l,\sigma}^{\text{Adam}} = \text{clip}(3.0 \max\{0.10, 1 + 1.20(\varrho_{\sigma}(l) - 1)\}, 0.40, 4.0). \quad (23)$$

The Muon branch is concentrated early in training and is reduced when relative scale imbalance or coefficient-gradient scale is high. Define

$$\Psi_{l,t} = \text{clip}\left(\frac{1}{G} \sum_{g=1}^G |\pi_{l,g}|, 0, 1.50\right), \quad \Omega_{l,t} = \text{ReLU}\left(\frac{G^{-1} \sum_{g=1}^G \alpha_{l,g} - 0.05}{0.45}\right), \quad (24)$$

$$\psi_{l,t} = \text{clip}(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15). \quad (25)$$

The branch gate is

$$\mu_{l,\sigma,t} = \text{clip}(0.75 \text{ smoothstep}(0.02, 0.12, \xi_t)[1 - \text{smoothstep}(0.20, 0.36, \xi_t)]\varrho_{\sigma}(l)\psi_{l,t}, 0, 0.75). \quad (26)$$

For $M_{l,\text{in}} = A_l$ and $M_{l,\text{out}} = B_l$, let η_t be the base learning rate. MatrixPolicy applies an AdamW matrix step followed by any active Muon substep. The gate $\mu_{l,\sigma,t}$ scales the two learning rates; it is not a convex mixing coefficient for a single update direction:

$$\begin{aligned} \tilde{M}_{l,\sigma}^{t+1} &= \text{AdamW}_{\eta_t s_{l,\sigma}^{\text{Adam}}(1-\mu_{l,\sigma,t})}(M_{l,\sigma}^t; \tilde{\nabla} M_{l,\sigma}^t), \\ M_{l,\sigma}^{t+1} &= \text{Muon}_{\eta_t \mu_{l,\sigma,t}}(\tilde{M}_{l,\sigma}^{t+1}; \tilde{\nabla} M_{l,\sigma}^t), \end{aligned} \quad (27)$$

The update in equation 27 is operational notation: AdamW and Muon carry separate optimizer states, their configured weight-decay and momentum terms are handled by the respective optimizer implementations, and both substeps use the gradient from training step t . Here Muon denotes the `torch.optim.Muon` update on two-dimensional RLB matrix parameters: momentum 0.95, five Newton–Schulz orthogonalization steps, and the match-RMS learning-rate adjustment of the Muon optimizer (Liu et al., 2025). When all Muon fractions have decayed to zero, this branch is inactive.

The last operation is a bounded positive rescaling of the two matrix blocks around each rational curve. It is evaluated after the AdamW/Muon matrix branch every five optimizer steps; on other steps it is skipped. Let $\mathcal{U} = \{u_k\}_{k=1}^{129}$ be a uniform probe grid over $[-5, 5]$, and define $\text{rms}_{\mathcal{U}}(\varphi) = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} \varphi(u_k)^2}$. The fixed-grid probe gains are diagnostics of the curve shape, not estimates of the current feature distribution. They are read inside the post-branch balancing operation when the probe metric is refreshed. The live gains in equation 14 provide the batch-distribution summaries for group-gradient redistribution. The probe gains are

$$\hat{d}_{l,g}^{\text{probe}} = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} R'_{l,g}(u_k)^2 + \epsilon_g}, \quad \hat{o}_{l,g}^{\text{probe}} = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} R_{l,g}(u_k)^2 + \epsilon_g}, \quad (28)$$

and the current matrix log ratio is computed from the post-branch matrix blocks with floored matrix RMS values,

$$\gamma_{l,g} = \log \text{rms}_{\epsilon_p}(A_{l,g}) - \log \text{rms}_{\epsilon_p}(B_{l,g}). \quad (29)$$

Because the floor is inside both logarithms, $\gamma_{l,g}$ is a stable implementation proxy for a matrix-RMS ratio. The target ratio combines curve-gain and relative-gradient terms,

$$\tau_{l,g} = \frac{1}{2} \left(\log \hat{d}_{l,g}^{\text{probe}} - \log \hat{o}_{l,g}^{\text{probe}} \right) + 0.25 \bar{\pi}_{l,g}, \quad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25). \quad (30)$$

The first term balances fixed-grid derivative and response gains in log space, with the factor 1/2 mapping the gain contrast to the matrix-pair log-ratio coordinate; the second term adds a smaller

pressure bias from the clipped RMS-imbalance proxy. Coefficient-gradient scale modulates the rescaling strength through

$$\chi_{l,g}^{rs} = \exp(0.10 \bar{\alpha}_{l,g}), \quad \bar{\alpha}_{l,g} = \text{clip}(\alpha_{l,g}, -1.25, 1.25). \quad (31)$$

The positive sign is intentional: high group coefficient activity damps the group-gradient path, while high layer-average coefficient activity damps the Muon branch, but the bounded balancing step moves slightly more quickly toward the representative ratio; the later log-step clip limits this effect. With schedule

$$s_{l,t} = 0.50 \text{ smoothstep}(0.03, 0.35, \xi_t) \text{ clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25), \quad (32)$$

the active-step log move is

$$\ell_{l,g,t} = \text{clip}\left(\frac{1}{2}[\pi_{l,g} - \gamma_{l,g}]s_{l,t}\chi_{l,g}^{rs}, -0.030, 0.030\right). \quad (33)$$

The matrix blocks are then updated as

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \quad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \quad (34)$$

The implementation applies the corresponding group scale to optimizer-state buffers as well, using squared scales for square-average state entries.

4 EXPERIMENTS

4.1 FIXED-SCALE TARGET-ARRIVAL STUDY

The fixed-scale study uses the same 12-layer, width-768 causal Transformer on five corpora: DCLM/DataComp-LM (Li et al., 2024), FineWeb-Edu and FineWeb (Penedo et al., 2024), Dolma-sample (Soldaini et al., 2024), and C4 (Raffel et al., 2020). The 100M-token budget uses 3050 optimizer steps and the 300M-token budget uses 9150 optimizer steps, with three seeds per method/dataset pair. The main comparison keeps the architecture and data fixed while changing the activation and optimizer used in the nonlinear sublayer: MatrixPolicy is compared with SiLU controls and with RLB-only controls, where RLB-only uses the same global-rational, no-local-atom activation but no MatrixPolicy updates. Shared optimizer, evaluation-cadence, and throughput-measurement details are reported in Appendix A.1; broader optimizer comparisons beyond AdamW and Muon are summarized in Table 2.

Table 1: Target-arrival efficiency for the fixed 12-layer, width-768 Transformer. RLB denotes the global-rational, no-local-atom activation without MatrixPolicy. At the hardest validation-loss target reached by every listed method in all three seeds, MatrixPolicy is always fastest; each token-saving column reports MatrixPolicy’s savings relative to the method named below it.

Budget	Dataset	Target loss	MatrixPolicy token savings (%)				Time to target (min)				
			vs. SiLU AdamW	vs. SiLU Muon	vs. RLB AdamW	vs. RLB Muon	Matrix Policy	SiLU AdamW	SiLU Muon	RLB AdamW	RLB Muon
100M	DCLM	4.55	20.7	32.4	20.7	34.3	12.8	20.8	26.0	14.3	19.0
	FineWeb-Edu	4.40	19.5	29.5	20.2	29.5	12.9	20.3	24.4	14.2	18.0
	FineWeb	4.60	22.8	32.1	21.5	32.1	12.6	16.4	20.2	16.2	20.1
	Dolma	4.60	22.0	34.0	22.7	34.9	12.8	17.6	22.0	17.3	22.2
	C4	4.60	20.7	29.2	19.3	28.7	11.6	14.3	17.3	14.9	18.8
300M	DCLM	4.10	23.3	8.3	24.0	7.0	39.6	53.5	45.9	54.7	45.6
	FineWeb-Edu	3.85	22.8	6.7	22.6	4.8	40.0	59.9	47.9	52.3	50.0
	FineWeb	4.10	23.1	7.1	22.4	5.3	40.8	61.2	46.4	55.6	46.6
	Dolma	3.95	22.3	9.8	22.3	6.5	44.4	49.3	52.3	50.5	47.4
	C4	4.00	26.4	8.5	25.3	6.5	41.8	64.7	55.5	59.4	51.9

Table 1 is the primary empirical result. It asks: for the hardest common validation-loss target reached by every listed method in all three seeds, what fraction of target-arrival tokens does MatrixPolicy

save, and how many measured target-arrival minutes does each method require? Across all five datasets under the 100M-token budget, MatrixPolicy saves 19.5–22.8% of the SiLU with AdamW target-arrival tokens and 29.2–34.0% of the SiLU with Muon target-arrival tokens. Under the 300M-token budget, after longer training compresses endpoint gaps, MatrixPolicy still saves 22.3–26.4% versus SiLU with AdamW and 6.7–9.8% versus SiLU with Muon, with lower target-arrival time in every listed comparison.

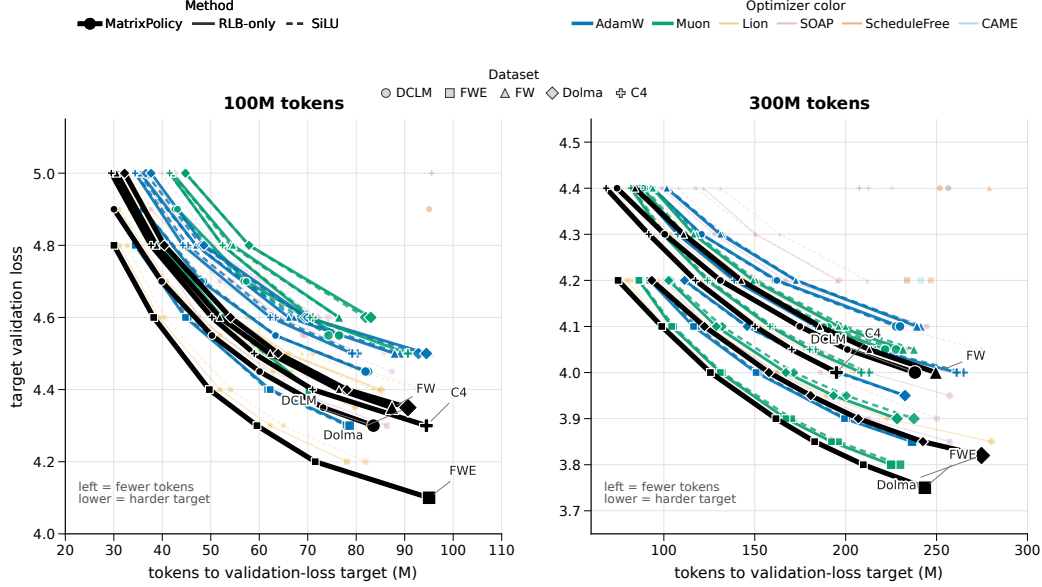


Figure 3: Target-arrival map for the completed fixed-scale study. Each point is the mean token count needed by one method to reach one validation-loss target on one dataset; connected points trace the same method and dataset from easier to harder targets. The horizontal coordinate is target-arrival tokens, and the vertical coordinate is the target validation loss, so curves further left reach the same loss with fewer tokens. The hard-target timing and token-saving percentages are quantified in Table 1.

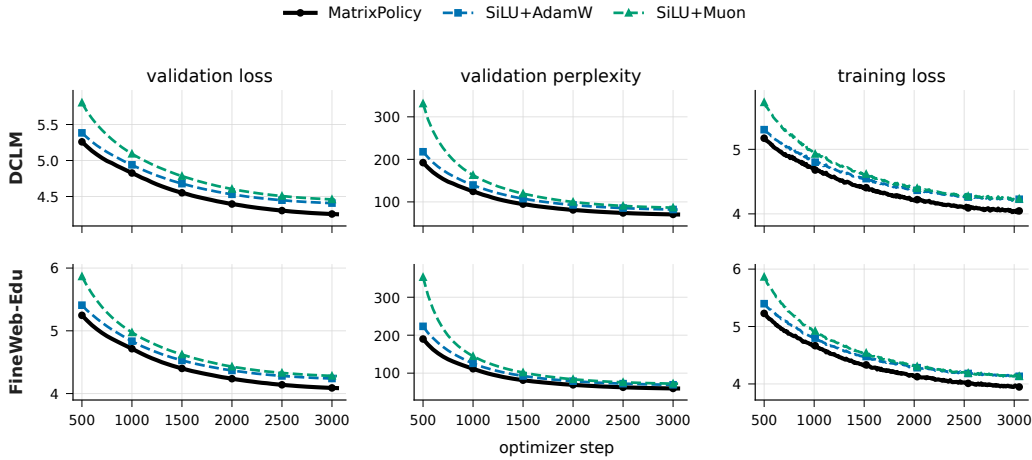


Figure 4: Representative 100M-token-budget training dynamics on DCLM and FineWeb-Edu. Rows show datasets and columns show validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation. This figure uses the two standard SiLU baselines so that the early target-arrival effect in Table 1 remains visually legible.

The 100M-token-budget curves show why target arrival is the right main readout. MatrixPolicy separates early in validation loss and validation perplexity while keeping training loss competitive. A long fixed budget eventually compresses many activation/optimizer endpoints, especially under the 300M-token budget, so the paper emphasizes the number of tokens and minutes needed to reach the same useful validation-loss levels rather than only the final loss after every method has consumed the full budget.

4.2 TODO: LARGER-SCALE TRAINING RESULTS

TODO: This subsection is reserved for larger-scale training results. The current claim is limited to the completed fixed-scale evidence in Section 4.1, Table 1, and Figures 3–4.

4.3 TODO: COMPONENT ABLATIONS

TODO: This subsection is reserved for component ablations isolating the centered group redistribution, transient Muon gate, bounded pair rescaling, and the global rational activation interface.

5 CONCLUSION

Global-rational RLB turns the nonlinear Transformer sublayer into an optimizer-visible interface: it exposes grouped normalized coordinates, trainable global rational responses, and an idealized positive matrix-pair relation used as a bounded balancing coordinate. MatrixPolicy uses that interface to update the RLB input and output matrices with role-aware statistics, while rational coefficients and non-RLB-matrix parameters use AdamW. The completed 100M-token and 300M-token runs show a consistent target-arrival pattern: MatrixPolicy reaches common validation losses with fewer tokens and lower measured target-arrival time than the listed SiLU and global-rational controls. Larger-scale training and component ablations remain explicit TODO subsections inside the Experiments section of the current draft.

REFERENCES

- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL <https://neurips.cc/virtual/2024/poster/96925>.
- Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. URL <https://arxiv.org/abs/1606.08415>.
- Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL <https://mlanthology.org/iclr/2015/kingma2015iclr-adam/>.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.
- Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zBPZeRjfgu>.

- Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is Scalable for LLM Training. *arXiv preprint arXiv:2502.16982*, 2025. URL <https://arxiv.org/abs/2502.16982>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL <https://aclanthology.org/2023.acl-long.243/>.
- Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for Activation Functions. *arXiv preprint arXiv:1710.05941*, 2017. URL <https://arxiv.org/abs/1710.05941>.
- Noam Shazeer. GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL <https://arxiv.org/abs/2002.05202>.
- Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. *arXiv preprint arXiv:2402.00159*, 2024. URL <https://arxiv.org/abs/2402.00159>.
- Maosen Tang and Alex Townsend. Rational Neural Networks have Expressivity Advantages. *arXiv preprint arXiv:2602.12390*, 2026. doi: 10.48550/arXiv.2602.12390. URL <https://arxiv.org/abs/2602.12390>.
- Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.
- Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=2J51qUZ0iG>.
- Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In *International Conference on Learning Representations*,

2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=hYHsrKDIX7>.

A EXPERIMENT APPENDIX

A.1 FIXED-SCALE TARGET-ARRIVAL STUDY

A.1.1 MODEL AND DATA PIPELINE

The fixed-scale experiments use a custom causal decoder Transformer rather than an off-the-shelf GPT-2 or LLaMA implementation. The block uses a LLaMA-style pre-normalized layout with RMSNorm before attention and before the nonlinear sublayer, bias-free query/key/value and output attention projections, rotary position embeddings, causal scaled-dot-product attention, residual attention and nonlinear-sublayer updates, a final RMSNorm, and a tied token-embedding/output head. The fixed model has 12 layers, width 768, 12 heads of dimension 64, and context length 256.

The SiLU controls use a SwiGLU-like gated nonlinear sublayer with bias-free matrices $W_g, W_v \in \mathbb{R}^{768 \times 2048}$ and $W_o \in \mathbb{R}^{2048 \times 768}$, computing $W_o(\text{SiLU}(xW_g) \odot xW_v)$. The global-rational RLB rows use a matched single-branch matrix pair: an input matrix maps width 768 to 3072 latent coordinates, the coordinates are partitioned into groups and transformed by the real-pole-free P5/Q4 global rational response, and an output matrix maps the 3072 coordinates back to width 768. This is the no-local-atom RLB variant used by the main MatrixPolicy results.

Tokenization uses the GPT-2 tokenizer through the Hugging Face tokenizer interface, without adding tokenizer-specific special tokens; an EOS or pad token is appended between documents. Training samples random contiguous blocks of length 257 and forms next-token prediction pairs of length 256. The DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4 runs use the text field specified in the manifests, with validation slices drawn from held-out token offsets of the same corpus stream.

A.1.2 TRAINING PROTOCOL

All runs use batch size 16, gradient accumulation 2, and therefore 32768 global tokens per optimizer step. The 100M-token budget consists of 3050 optimizer steps; the 300M-token budget consists of 9150 optimizer steps. Each method/dataset pair is run with seeds 1337, 2027, and 3407. Validation is performed every 50 optimizer steps, so token-to-target arrivals are quantized in increments of 1.64M tokens.

The MatrixPolicy rows in the main table use the global-rational RLB activation with no local atoms: the active nonlinearity is only the group-global P5/Q4 response in equation 5–equation 6. The reported configuration uses AdamW as the backbone optimizer and disables the separate rational-coefficient optimizer. It enables MatrixPolicy group redistribution with gain/pressure/activity strengths 0.20/0.10/0.20, schedule window 0.02–0.30, and group multiplier clip [0.75, 1.35]. The bounded RLB pair-rescale wrapper runs every five optimizer steps with covariant optimizer state scaling. Rational coefficients, attention weights, embeddings, normalization parameters, and ordinary Transformer weights use AdamW in this configuration.

Target-arrival minutes in Table 1 are computed seed-wise from the first validation event that reaches the shared target and the same run’s measured seconds per optimizer step.

A.1.3 SUPPORTING FIXED-SCALE CURVES

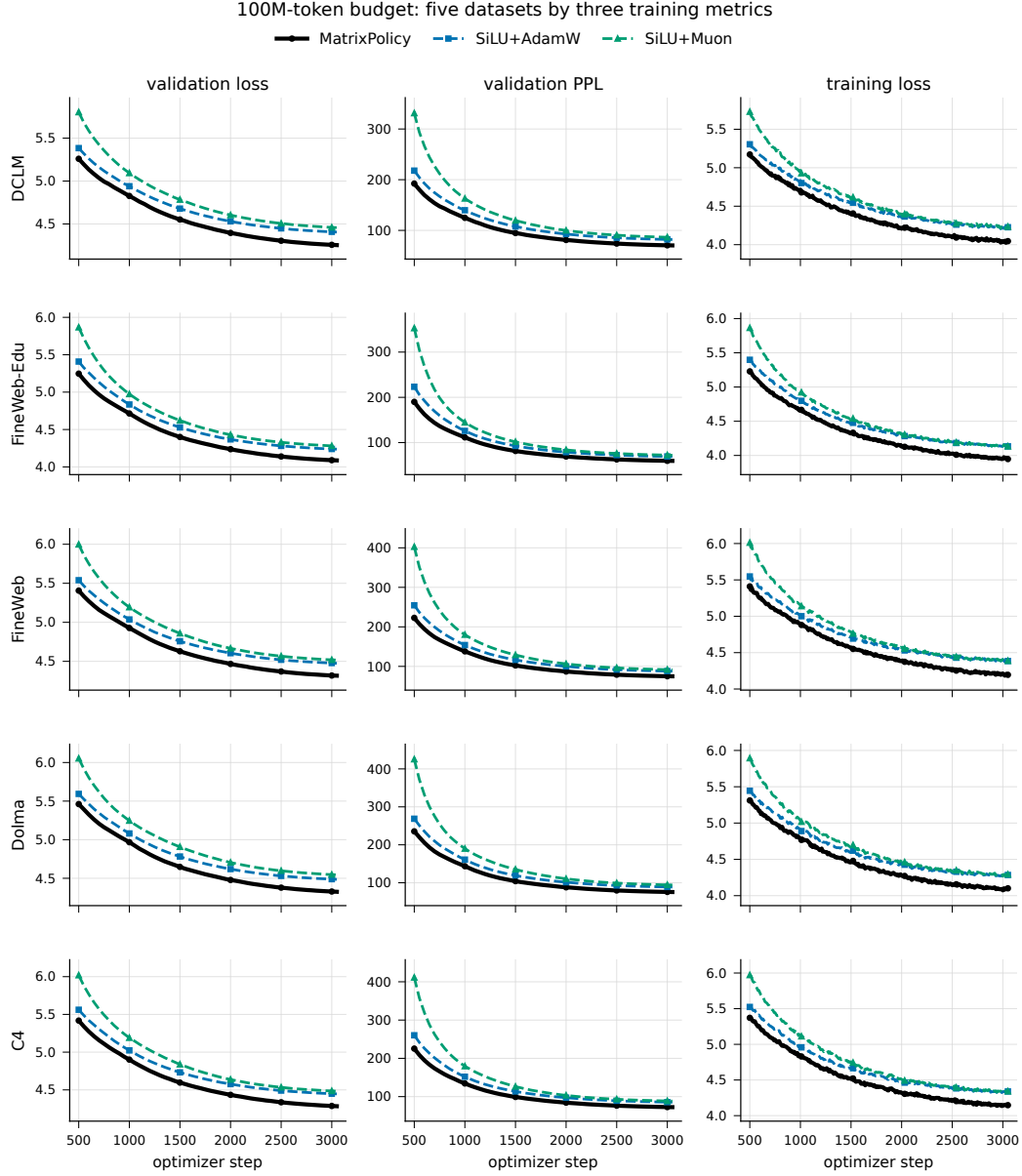


Figure 5: 100M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

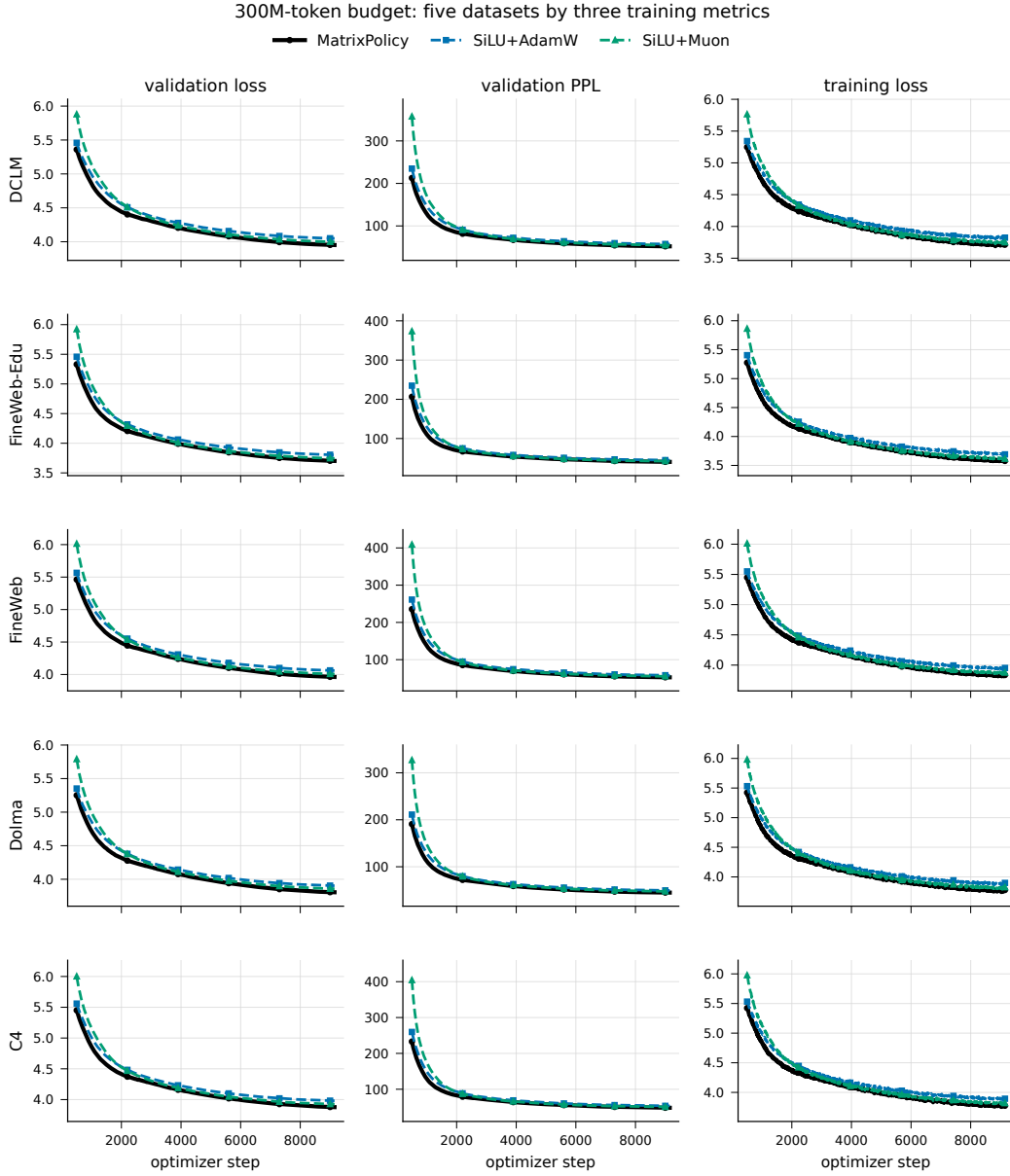


Figure 6: 300M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

Table 2: Final validation loss for the broader optimizer sweep on the fixed 12-layer, width-768 language model. Values are mean $\pm$ sample standard deviation over three seeds; lower is better. RLB rows use the global-rational, no-local-atom activation without MatrixPolicy.

Budget	Method	DCLM	FineWeb-Edu	FineWeb	Dolma	C4
100M tokens	MatrixPolicy	4.2538 $\pm$ 0.0063	4.0873 $\pm$ 0.0102	4.3162 $\pm$ 0.0125	4.3253 $\pm$ 0.0053	4.2837 $\pm$ 0.0197
	SiLU + AdamW	4.4056 $\pm$ 0.0099	4.2375 $\pm$ 0.0086	4.4758 $\pm$ 0.0097	4.4862 $\pm$ 0.0012	4.4469 $\pm$ 0.0160
	SiLU + Lion	4.3183 $\pm$ 0.0069	4.1494 $\pm$ 0.0092	4.3825 $\pm$ 0.0083	4.3878 $\pm$ 0.0046	4.3536 $\pm$ 0.0156
	SiLU + Muon	4.4572 $\pm$ 0.0126	4.2787 $\pm$ 0.0243	4.5163 $\pm$ 0.0264	4.5443 $\pm$ 0.0108	4.4824 $\pm$ 0.0233
	SiLU + SOAP	4.4160 $\pm$ 0.0038	4.2630 $\pm$ 0.0220	4.4840 $\pm$ 0.0112	4.4987 $\pm$ 0.0035	4.4582 $\pm$ 0.0146
	SiLU + ScheduleFree	4.9023 $\pm$ 0.0111	4.8258 $\pm$ 0.0072	5.0142 $\pm$ 0.0188	5.0494 $\pm$ 0.0149	5.0064 $\pm$ 0.0160
	SiLU + CAME	5.0107 $\pm$ 0.0143	4.9207 $\pm$ 0.0123	5.1325 $\pm$ 0.0126	5.1650 $\pm$ 0.0189	5.1230 $\pm$ 0.0178
	RLB + AdamW	4.4046 $\pm$ 0.0081	4.2392 $\pm$ 0.0077	4.4717 $\pm$ 0.0138	4.4878 $\pm$ 0.0027	4.4412 $\pm$ 0.0162
	RLB + Lion	4.2946 $\pm$ 0.0083	4.1361 $\pm$ 0.0083	4.3626 $\pm$ 0.0112	4.3622 $\pm$ 0.0066	4.3271 $\pm$ 0.0160
	RLB + Muon	4.4674 $\pm$ 0.0121	4.2831 $\pm$ 0.0284	4.5157 $\pm$ 0.0059	4.5476 $\pm$ 0.0153	4.4764 $\pm$ 0.0099
	RLB + SOAP	4.4360 $\pm$ 0.0442	4.2747 $\pm$ 0.0238	4.6893 $\pm$ 0.3359	4.5183 $\pm$ 0.0363	4.4458 $\pm$ 0.0190
	RLB + ScheduleFree	4.8871 $\pm$ 0.0046	4.7957 $\pm$ 0.0133	4.9993 $\pm$ 0.0192	5.0363 $\pm$ 0.0129	4.9842 $\pm$ 0.0164
	RLB + CAME	5.0139 $\pm$ 0.0086	4.9150 $\pm$ 0.0063	5.1340 $\pm$ 0.0162	5.1514 $\pm$ 0.0168	5.1109 $\pm$ 0.0155
300M tokens	MatrixPolicy	3.9518 $\pm$ 0.0282	3.7015 $\pm$ 0.0212	3.9623 $\pm$ 0.0081	3.8062 $\pm$ 0.0073	3.8777 $\pm$ 0.0144
	SiLU + AdamW	4.0493 $\pm$ 0.0275	3.8035 $\pm$ 0.0182	4.0612 $\pm$ 0.0101	3.9037 $\pm$ 0.0091	3.9811 $\pm$ 0.0128
	SiLU + Lion	3.9934 $\pm$ 0.0230	3.7440 $\pm$ 0.0208	4.0015 $\pm$ 0.0085	3.8475 $\pm$ 0.0094	3.9213 $\pm$ 0.0105
	SiLU + Muon	3.9973 $\pm$ 0.0305	3.7454 $\pm$ 0.0170	4.0066 $\pm$ 0.0128	3.8581 $\pm$ 0.0101	3.9251 $\pm$ 0.0134
	SiLU + SOAP	4.0964 $\pm$ 0.0300	3.8629 $\pm$ 0.0201	4.1139 $\pm$ 0.0101	3.9568 $\pm$ 0.0093	4.0349 $\pm$ 0.0108
	SiLU + ScheduleFree	4.3657 $\pm$ 0.0298	4.1559 $\pm$ 0.0238	4.3979 $\pm$ 0.0106	4.2151 $\pm$ 0.0053	4.3163 $\pm$ 0.0107
	SiLU + CAME	4.3682 $\pm$ 0.0226	4.1503 $\pm$ 0.0211	4.4060 $\pm$ 0.0189	4.2492 $\pm$ 0.0270	4.3298 $\pm$ 0.0148
	RLB + AdamW	4.0496 $\pm$ 0.0298	3.8024 $\pm$ 0.0206	4.0601 $\pm$ 0.0110	3.9045 $\pm$ 0.0082	3.9787 $\pm$ 0.0098
	RLB + Lion	3.9887 $\pm$ 0.0295	3.7416 $\pm$ 0.0214	3.9960 $\pm$ 0.0105	3.8412 $\pm$ 0.0085	3.9132 $\pm$ 0.0139
	RLB + Muon	3.9913 $\pm$ 0.0260	3.7373 $\pm$ 0.0187	3.9991 $\pm$ 0.0110	3.8478 $\pm$ 0.0047	3.9189 $\pm$ 0.0149
	RLB + SOAP	4.0608 $\pm$ 0.0331	3.8240 $\pm$ 0.0128	4.1081 $\pm$ 0.0320	3.9269 $\pm$ 0.0125	4.0024 $\pm$ 0.0194
	RLB + ScheduleFree	4.3607 $\pm$ 0.0344	4.1421 $\pm$ 0.0217	4.3864 $\pm$ 0.0081	4.2121 $\pm$ 0.0112	4.3084 $\pm$ 0.0071
	RLB + CAME	4.4503 $\pm$ 0.0426	4.2255 $\pm$ 0.0358	4.4804 $\pm$ 0.0064	4.2665 $\pm$ 0.0409	4.3651 $\pm$ 0.0582

A.2 TODO: LARGER-SCALE TRAINING RESULTS

TODO: This appendix subsection is reserved for the larger-scale training results that will complete Section 4.2. No result is reported here until the corresponding experiment is complete.

A.3 TODO: COMPONENT ABLATIONS

TODO: This appendix subsection is reserved for the component ablations that will complete Section 4.3. The planned ablations isolate the centered group redistribution, transient matrix-direction branch, bounded matrix-pair rescaling, and global-rational activation interface.

B RATIONAL LATENT BASIS CALCULUS

Real-pole-free group-global response. For one layer and group, write

$$\begin{aligned} \phi_1(u) &= |u|, \quad \phi_2(u) = u^2, \quad \phi_3(u) = |u|^3, \quad \phi_4(u) = u^4, \\ P(u) &= \sum_{i=0}^5 a_i u^i, \quad Q(u) = 1 + \sum_{j=1}^4 |b_j| \phi_j(u). \end{aligned} \quad (35)$$

Since $\phi_j(u) \geq 0$ for all real u , $Q(u) \geq 1$ and $R(u) = P(u)/Q(u)$ has no real pole. Away from $u = 0$ for input derivatives,

$$R'(u) = \frac{P'(u)Q(u) - P(u)Q'(u)}{Q(u)^2}, \quad (36)$$

where

$$P'(u) = \sum_{i=1}^5 i a_i u^{i-1}, \quad Q'(u) = |b_1| \text{sign}(u) + 2|b_2|u + 3|b_3|u|u| + 4|b_4|u^3. \quad (37)$$

The implementation uses the autodiff convention $\text{sign}(0) = 0$ for derivative-gain statistics. The coefficient features are explicit:

$$\frac{\partial R(u)}{\partial a_i} = \frac{u^i}{Q(u)}, \quad \frac{\partial R(u)}{\partial b_j} = -\frac{P(u)}{Q(u)^2} \text{sign}(b_j) \phi_j(u), \quad (38)$$

with the usual autodiff subgradient convention at $b_j = 0$. These derivatives make output-factor and derivative gains directly observable from the layer/group global curve, without introducing local-atom parameters.

Normalized-group Jacobian and gain summaries. For a group vector $z \in \mathbb{R}^m$, define

$$r(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}, \quad u(z) = z/r(z), \quad h(z) = r(z)R(u(z)), \quad (39)$$

where R is applied coordinatewise. The radius derivative is

$$\text{d}r = \frac{z^\top \text{d}z}{mr} = \frac{u^\top \text{d}z}{m}, \quad (40)$$

and the normalized-coordinate derivative is

$$\text{d}u = \frac{1}{r} \left(I - \frac{uu^\top}{m} \right) \text{d}z. \quad (41)$$

Therefore the group Jacobian is

$$J_h(z) := \frac{\partial h}{\partial z} = R(u) \frac{u^\top}{m} + \text{diag}(R'(u)) \left(I - \frac{uu^\top}{m} \right). \quad (42)$$

This decomposition motivates separate output-gain and derivative-gain summaries. The vector $R(u)$ controls the curve-response factor inside the realized feature $h = rR(u)$, while $R'(u)$ appears in the tangent component of the input Jacobian seen by A_l . The response and derivative RMS summaries used by MatrixPolicy are not full feature-scale or input-Jacobian measurements: the full feature also depends on r , and the full input gradient also depends on the radial $R(u)u^\top/m$ term and on $B_{l,g}$.

Positive matrix-pair rescaling with an RMS floor. Set $\epsilon_{\text{rms}} = 0$ and take $z \neq 0$ and $\lambda > 0$. Then

$$r(\lambda z) = \lambda r(z), \quad u(\lambda z) = u(z), \quad h(\lambda z) = \lambda h(z). \quad (43)$$

Scaling $A_{l,g}$ by λ and the matching column block $B_{l,g}$ by λ^{-1} therefore leaves the group contribution unchanged. Applying this independently to all groups gives equation 9.

With $\epsilon_{\text{rms}} > 0$, define the floored maps

$$r_\epsilon(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}, \quad u_\epsilon(z) = z/r_\epsilon(z), \quad h_\epsilon(z) = r_\epsilon(z)R(u_\epsilon(z)). \quad (44)$$

For $\rho^2 = m^{-1}\|z\|_2^2$,

$$u_\epsilon(\lambda z) = \kappa(\lambda, z)u_\epsilon(z), \quad \kappa(\lambda, z) = \frac{\lambda\sqrt{\rho^2 + \epsilon_{\text{rms}}}}{\sqrt{\lambda^2\rho^2 + \epsilon_{\text{rms}}}}. \quad (45)$$

Let L_R be a Lipschitz constant of the scalar curve over an interval containing all coordinates of $u_\epsilon(z)$ and $\kappa(\lambda, z)u_\epsilon(z)$. After compensating the output matrix by λ^{-1} , the group-feature error obeys

$$\|\lambda^{-1}h_\epsilon(\lambda z) - h_\epsilon(z)\|_2 \leq \left| \frac{r_\epsilon(\lambda z)}{\lambda} - r_\epsilon(z) \right| \|R(u_\epsilon(z))\|_2 \quad (46)$$

$$+ \frac{r_\epsilon(\lambda z)}{\lambda} L_R |\kappa(\lambda, z) - 1| \|u_\epsilon(z)\|_2. \quad (47)$$

Both terms vanish when $\epsilon_{\text{rms}} = 0$. When $\rho^2 \gg \epsilon_{\text{rms}}$ and the global response and derivative are bounded on the interval above, the bound is small; near the floor, the approximation can be poor. This is the reason positive matrix-pair balancing is treated as a bounded log-coordinate move rather than an unrestricted change of representative.

C MATRIXPOLICY UPDATE GEOMETRY

Optimizer step order. At training step t , backpropagation produces gradients with respect to the pre-update parameters. MatrixPolicy computes the pressure and activity statistics from these gradients and from A_l^t , B_l^t , and the rational coefficients before any parameter is changed. With EMA decay $\beta = 0.95$, the matrix and coefficient statistics have the form

$$q_{l,g}^{\text{in},t} = \beta q_{l,g}^{\text{in},t-1} + (1 - \beta) p_{l,g}^{\text{in},t}, \quad (48)$$

$$q_{l,g}^{\text{out},t} = \beta q_{l,g}^{\text{out},t-1} + (1 - \beta) p_{l,g}^{\text{out},t}, \quad (49)$$

$$q_{l,g}^{R,t} = \beta q_{l,g}^{R,t-1} + (1 - \beta) p_{l,g}^{R,t}. \quad (50)$$

The live curve gains are read from the RLB forward cache that produced the same loss and enter the group-gradient multipliers in equation 16. The Muon fraction in equation 26 and the balancing quantities in equation 30–equation 31 use pre-update pressure/activity EMAs collected by the optimizer wrapper. Non-matrix Transformer parameters and rational coefficients then follow the AdamW path. The RLB matrix blocks use the scaled gradients in equation 21 for the AdamW and Muon matrix branches in equation 27. On scheduled rescaling steps, the bounded positive matrix-pair move in equation 34 is applied after the matrix branch, and the corresponding optimizer-state buffers are scaled covariantly. Thus MatrixPolicy changes only the representatives of RLB input/output matrix pairs; it does not introduce a separate optimizer path for rational coefficients or for non-RLB tensors.

MatrixPolicy is built from the quantities that the RLB parameterization exposes without adding a new optimization objective. The normalized group map separates curve response from tangent sensitivity; the positive rescaling direction creates a near-invariant coordinate for the input/output matrix pair away from the RMS floor; and the relative-gradient summaries provide a cheap estimate of which role is changing faster. These three signals determine where MatrixPolicy redistributes group update scale, how strongly it moves the two matrix representatives toward a balanced log ratio, and when it applies the transient matrix-direction branch.

Role-specific gradients and gain selection. For one training example, let $\bar{y}_l = \nabla_{y_l} \mathcal{L}$ be the upstream gradient, and let $J_{l,g} = \partial h_{l,g} / \partial z_{l,g}$ be the group Jacobian in equation 42. The adjacent matrix gradients factor as

$$\nabla_{B_{l,g}} \mathcal{L} = \bar{y}_l h_{l,g}^\top, \quad (51)$$

$$\nabla_{A_{l,g}} \mathcal{L} = (J_{l,g}^\top B_{l,g}^\top \bar{y}_l) x_l^\top. \quad (52)$$

The output matrix is coupled directly to the realized feature $h_{l,g}$, whereas the input matrix is coupled to the tangent and radial components of the RLB Jacobian before the upstream gradient reaches x_l . This asymmetry motivates separate gain summaries: response RMS summaries act as output-role proxies, and derivative RMS summaries act as input-role proxies. These summaries are partial by construction. The full output feature also contains $r_{l,g}$, and the full input gradient also depends on $B_{l,g}$ and on the radial term $R(u)u^\top/m$ in equation 42. MatrixPolicy therefore uses gains as bounded role signals for the adjacent matrix updates.

Scale-normalized pressure and centered redistribution. In the idealized unfloored rescaling direction, an infinitesimal increase of the input representative and reciprocal decrease of the output representative has the loss derivative

$$\Delta_g = \langle \nabla_{A_{l,g}} \mathcal{L}, A_{l,g} \rangle_F - \langle \nabla_{B_{l,g}} \mathcal{L}, B_{l,g} \rangle_F. \quad (53)$$

MatrixPolicy does not estimate Δ_g directly. Instead it uses RMS relative gradient scales because they are cheap and stable. For nonzero unfloored W , if $W^+ = W - \eta \mathcal{G}$ and η is small, then

$$\log \text{rms}(W^+) - \log \text{rms}(W) = -\eta \frac{\langle \mathcal{G}, W \rangle_F}{\|W\|_F^2} + O(\eta^2), \quad (54)$$

and Cauchy–Schwarz implies

$$\left| \frac{\langle \mathcal{G}, W \rangle_F}{\|W\|_F^2} \right| \leq \frac{\|\mathcal{G}\|_F}{\|W\|_F} = \frac{\text{rms}(\mathcal{G})}{\text{rms}(W)}. \quad (55)$$

The implemented quantities p^{in} and p^{out} are floored versions of the upper-bound scale on the right-hand side. They are unsigned magnitudes. Thus $\pi_{l,g} = \log q_{l,g}^{\text{in}} - \log q_{l,g}^{\text{out}}$ is an RMS-imbalance proxy: its sign identifies which role currently has the larger relative update scale, and its clipped magnitude supplies the pressure term used by equation 18 and equation 30.

The redistribution step is centered separately for each role. Before clipping,

$$\hat{c}_{l,g,\sigma} = \frac{c_{l,g,\sigma}}{\text{geomean}_{j=1}^G c_{l,j,\sigma}}, \quad \frac{1}{G} \sum_{g=1}^G \log \hat{c}_{l,g,\sigma} = 0. \quad (56)$$

The policy therefore redistributes update scale across groups while preserving the role-wise mean log multiplier before clipping. The clip in equation 20 deliberately trades exact centering for robustness to noisy group statistics.

Bounded matrix-pair balancing and transient direction updates. For the unfloored nonzero-matrix calculation, write

$$\gamma_{l,g} = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (57)$$

An active rescaling step applies $A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}$ and $B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}$, hence

$$\gamma_{l,g}^+ = \gamma_{l,g} + 2\ell_{l,g,t}. \quad (58)$$

If clipping is ignored and $\lambda_{l,g,t} = s_{l,t} \chi_{l,g}^{\text{rs}}$ uses the schedule and activity factor from equation 31–equation 32, then equation 33 gives

$$\gamma_{l,g}^+ = (1 - \lambda_{l,g,t}) \gamma_{l,g} + \lambda_{l,g,t} \tau_{l,g}. \quad (59)$$

This is an exact contraction only in the nonzero-matrix, unfloored-RMS, $\epsilon_{\text{rms}} = 0$, unclipped idealization. The implementation uses the floored proxy in equation 29; near the floor, the log-ratio interpretation and the contraction model are approximate.

The target

$$\tau_{l,g} = \frac{1}{2} \left(\log \hat{d}_{l,g}^{\text{probe}} - \log \hat{o}_{l,g}^{\text{probe}} \right) + 0.25 \bar{\pi}_{l,g}, \quad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25) \quad (60)$$

uses fixed-grid probe gains because pair rescaling is meant to balance the representative around the curve shape rather than chase minibatch feature noise. The derivative/response contrast supplies the log-space gain-balancing target, the factor 1/2 maps that contrast to the matrix-pair ratio coordinate in equation 58, and the smaller pressure term biases the target toward the role with the larger relative update scale.

The time window inside the Muon fraction in equation 26 is

$$w(\xi_t) = \text{smoothstep}(0.02, 0.12, \xi_t) [1 - \text{smoothstep}(0.20, 0.36, \xi_t)]. \quad (61)$$

Together with the role factor $\varrho_\sigma(l)$, penalty $\psi_{l,t}$, and clipping in the full $\mu_{l,\sigma,t}$ formula, this window confines the matrix-direction update to the phase in which changes to latent directions most directly affect the coordinates seen by the rational curves. The penalty $\psi_{l,t}$ reduces this branch when RMS imbalance or coefficient-gradient activity is high. MatrixPolicy therefore uses Muon as a bounded, state-dependent matrix-direction substep rather than as a persistent generic optimizer for all tensors.